```
describe('Movies API', () => {
  describe('When a GET request is made to /movies', () => {
    test('it should return all movies', () => {

 #1
    });
  });
});
```

#1 Our first test here

A `describe` block in Jest is a way to organize the structure of your test. The top-level `describe` block explains that the test is for the movies API. The second `describe` block is for the specific scenario we want to test, which is scenario 1. For scenario 1, the consumer expects that when a `GET` request is made to the `/movies/` endpoint, the provider should respond with all the movies. The `test` block represents the actual test itself. To register a new expectation or interaction, we need to set up the mock provider with the correct consumer expectations in Pact by using the following code:

```
test('it should return all movies', () => {
   provider
     .uponReceiving('a request to all movies')
     .withRequest({
        method: 'GET',
        path: '/movies',
      })
      .willRespondWith({
        status: 200,
        body: MatchersV3.eachLike(EXPECTED_BODY),
      });
    });
 });
```

With this interaction, we're calling the `uponReceiving` API and passing in the scenario name as a string. It's a recommended practice to keep the naming easy to understand. In this example, we're saying that the scenario name is `'a request to all movies'`. Next, we call the `withRequest` API and pass in the HTTP request information, which, in this case, is a `GET` request to the `'/movies'` endpoint.

To represent the HTTP response details from the provider, we call the `willRespondWith` API and pass in the response details. In this example, the provider should respond with a status of 200. The response body is represented as a variable called `EXPECTED_BODY`. Let's first declare this variable outside our test: